

Железо для ИИ: от задачи к конфигурации

Покупка начинается с измеримой работы. Одна и та же модель предъявляет разные требования при одиночном разговоре, многопользовательском сервисе и дообучении.

Что именно должна сделать машина

В этом выпуске построим локальную систему для работы с документами: загрузка PDF и таблиц, OCR, извлечение фрагментов, поиск, переранжирование и ответ с цитатами. Иногда владелец хочет дообучить генератор или модель эмбедингов. Такая постановка полезнее фразы «мне нужна машина для 8B»: число параметров не сообщает ни длины контекста, ни числа одновременных обращений, ни скорости индексации.

Сначала составим паспорт нагрузки. Укажем точные ревизии генератора, OCR, embedding и reranker; количество и тип документов; распределение длин запроса и ответа; число одновременно активных генераций. Определим допустимое время до первого токена, скорость дальнейшей выдачи и качество ответов с правильными страницами источников. Отдельно запишем, что данные должны оставаться на собственной машине. Эти ограничения превращаются в условия, по которым можно испытать конфигурацию.

Пять путей через одну систему

При индексации CPU разбирает форматы и очередь, OCR может использовать GPU, SSD подаёт страницы, а RAM удерживает промежуточные тексты. Embedding-модель превращает фрагменты в векторы; база хранит их вместе с метаданными. Запрос пользователя сначала ищет кандидатов, затем генератор получает выбранные фрагменты. *Prefill* обрабатывает входные токены, *decode* создаёт выходные по одному. Обучение добавляет градиенты, активации, оптимизатор и запись checkpoint.

GPU важен не на каждой стадии. Медленный парсер или база, занятая восстановлением индекса, не ускорятся от более быстрой видеокарты. И наоборот, достаточный CPU не компенсирует нехватку памяти под веса и активные последовательности. Поэтому паспорт делится на пять строк: подготовка корпуса, поиск, одиночный инференс, сервис под нагрузкой и адаптация модели. Для каждой строки нужны своя единица результата и своё ограничение по времени.

Вместимость и производительность

«Загрузилось» означает лишь, что размещение удалось в заданном режиме. Часть весов может находиться в RAM или на диске, а кэш при реальных запросах ещё не выделен. «Годится» означает, что точная задача работает при целевых задержке, качестве и устойчивости. Это различие проведём через все расчёты выпуска.

Сквозной пример

Документная система с генератором, embedding, OCR и индексом. Первое испытание: один пользователь; затем двадцать одновременных последовательностей по 4096 токенов.

Паспорт нагрузки

Вход

Ревизия модели, документы, токены, частота запросов.

Выход

Качество ответа, ссылки, задержка, число завершённых запросов.

Граница

Локальность данных, деньги, энергия, время восстановления.

Разные слова

Пользователи системы, активные запросы и удерживаемые KV-последовательности не равны друг другу.

От паспортных чисел к узкому месту

CUDA Programming Guide¹ описывает иерархию памяти и исполнение ядер. Внутри ускорителя вычисления могут опережать подачу данных. Best Practices Guide² учит измерять фактические передачи между CPU и GPU. Roofline³ предлагает модель: если для одного байта перемещаемых данных выполняется мало операций, верхняя граница задаётся bandwidth; при большей интенсивности — вычислительным пиком. Формально, для ядра с интенсивностью I производительность ограничена $\min(P_{\max}, I B_{\max})$. Это верхняя граница при заданных предположениях, а не обещание скорости целого LLM-сервера. FlashAttention⁴ показывает, что способ перемещения данных может быть важнее числа арифметических действий: промежуточные матрицы не нужно полностью записывать в дорогую память. Но и это не означает одинакового выигрыша при коротком входе, другой архитектуре или неподдерживаемом ядре. По той же причине таблица FLOPS не даёт прямой ранжировки машин для интерактивной генерации.

Как выбирать между классами машин

Для одиночного исследования начните с имеющейся машины и небольшой открытой модели. Если веса с рабочим кэшем помещаются в доступную память и результат отвечает требованиям, рост класса GPU необязателен. При большом числе параллельных запросов проверьте суммарный KV и p95 задержки. При редком полном обучении сравните постоянную покупку кластера с арендой на время эксперимента. Локальный RAG может больше выиграть от памяти и надёжного SSD под корпус, чем от добавления второго GPU.

Решение оформляется проверяемым протоколом: версия модели и квантизации, версия программного стека, размеры входов, распределение поступления запросов, измеренные задержки и качество. Цена покупки — только одна часть стоимости; добавляются электричество, хранение, эксплуатация и возможный простой. Здесь не даём готовой корзины устройств, потому что совместимость и рыночная цена меняются. Вместо этого в конце выпуска читатель составит три допустимых конфигурации и протестирует точные модели.

Маршрут выпуска

Сначала разберём движение данных и памяти. Затем оценим веса и KV, расходы обучения, работу сервера, индекс и локальный запуск. Последняя часть сведёт расчёты к выбору CPU, GPU, RAM, SSD, платы, питания и сети. Каждый вывод будет иметь три уровня: нижняя оценка по формуле, проверка поддержки и фактический замер на задаче.

Roofline для ядра

$P \leq \min(P_{\max}, I B_{\max})$. Нужно знать и операции, и байты, реально перемещаемые на рассматриваемом уровне памяти.

Четыре результата

Помещается

Веса и рабочее состояние выделяются.

Запускается

Все операции поддержаны.

Быстро

Задержки удовлетворяют задаче.

Приемлемо

Качество и полная стоимость тоже подходят.

Первое чтение

CUDA¹: память и исполнение. Best Practices²: фактические передачи. Roofline³: границы. FlashAttention⁴: алгоритм движения данных.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

4. Dao, T. и др. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.

FlashAttention: точное внимание с учётом обмена между уровнями памяти

Разбиение вычисления attention на блоки уменьшает движение данных между HBM и быстрой памятью GPU. Это пример ускорения без изменения математической цели attention; эффект зависит от формы задачи и реализации.

Что читать. Постановка IO-aware attention и раздел Algorithm; объяснить, какие промежуточные данные перестают записываться в HBM.

Порядок чтения. CUDA даёт аппаратный словарь, Best Practices — измерение передач. Затем построить простой Roofline для одного ядра. Завершить FlashAttention как пример изменения алгоритма обращения к памяти. После чтения описать узкое место собственного процесса в проверяемой форме.

1. Архитектура узла и иерархия памяти

CPU, RAM, ускоритель, SSD и сеть образуют путь данных. Баланс этих звеньев зависит от того, какие операции выполняются и где находятся данные.

Машина — это путь, а не список деталей

CPU управляет процессами, токенизацией, парсингом, очередями и частью численных операций. RAM удерживает документы, индексные структуры, модель при выгрузке и файловый кэш. GPU быстро выполняет большие массивы операций, используя собственную VRAM или общую память в некоторых системах. SSD предоставляет checkpoint и корпус; сеть доставляет документы, запросы и данные при распределённом обучении. У каждого перехода есть задержка и пропускная способность.

Рассмотрим запрос к документной системе. Даже если матричные умножения занимают миллисекунды, поиск может ждать страницы с диска, а передача результата reranker в контекст генератора — CPU. Большой объём RAM полезен, когда индекс и рабочий корпус помещаются в ней; большая VRAM нужна, когда мы удерживаем веса и KV на GPU. Подмена одной памяти другой возможна только если runtime поддерживает перенос и задержка остаётся приемлемой.

CPU и системная память

Для CPU сравнивайте число ядер, производительность одного потока, поддерживаемые инструкции, каналы RAM и пропускную способность памяти. OCR и загрузка многих документов могут хорошо распараллеливаться, тогда как синхронный участок запроса ограничивает общую задержку. Для RAM важны ёмкость, bandwidth, возможность расширения и, в сервере, требования к ECC. NUMA означает, что доступ к памяти другого сокета может стоить дороже локального; размещение процесса и данных имеет значение.

При выборе SSD запишите размер исходного корпуса, производных фрагментов, индекса, моделей, журналов и резервных копий. Последовательная скорость из рекламы не описывает чтение множества мелких файлов или восстановление после сбоя. Отдельный ресурс нужен для временных преобразований и свободного места при пересоздании индекса. Измерения делайте после прогрева и при типичном количестве одновременных задач.

GPU и размещение тензоров

Руководство CUDA¹ различает регистры, быструю память на чипе и глобальную память устройства. VRAM ёмкость отвечает на вопрос о размещении, её bandwidth — на вопрос о подаче данных, а вычислительные блоки и поддерживаемые численные форматы — о скорости подходящих ядер. Ширина шины в битах сама по себе не заменяет измеренную пропускную способность памяти.

Путь одного запроса

SSD/RAM: документы и индекс.
CPU: поиск и токенизация.
GPU: prefill и decode. CPU/сеть: выдача и журнал.

Три свойства GPU

Ёмкость

Весы, KV и буферы.

Bandwidth

Подача весов и кэша.

Вычисления

Операции в поддерживаемом формате.

Вопрос при покупке

Есть ли на материнской плате нужные PCIe-линии, слоты RAM и место для SSD при установке выбранного ускорителя?

Передачи и реальные пределы

CUDA Best Practices Guide² предлагает отдельно смотреть host-device transfer и локальность доступов. Если при decode часть слоёв лежит в RAM, эти веса снова проходят через узкий канал или исполняются на CPU. Иногда такой режим делает модель доступной, но худшее время ответа меняет пригодность машины. В кластере аналогичную роль играет межсоединение GPU и сеть: наличие двух ускорителей не означает, что произвольный checkpoint автоматически исполняется вдвое быстрее.

Roofline³ помогает классифицировать операцию. У prefill крупные матричные операции могут эффективно использовать вычислительные блоки. При одиночном decode полезная работа на загружаемый блок весов меньше, поэтому memory bandwidth часто важнее. Это качественное правило, а не универсальная диагностика: размер batch, архитектура, квантование и реализация ядра меняют интенсивность.

Питание, охлаждение и физическая совместимость

Проверка железа не заканчивается названием GPU. Для настольной машины запишите физическую длину и толщину карты, расположение разъёмов питания, мощность БП с запасом под длительную нагрузку и возможность отвести тепло от GPU, CPU, RAM и SSD. Для сервера добавьте устойчивость питания, дистанционное управление, сетевые порты и путь замены диска. Машина, которая выдерживает пятиминутный тест, может снижать частоты через час или падать при долгом обучении.

Проверяйте *совместимость стека*: ОС, драйвер, runtime, версия библиотек, поддержка формата весов и операций конкретной модели. Если аппаратное устройство существует в матрице поставщика, но требуемое ядро отсутствует в выбранном приложении, задача не решена. Зафиксируйте версии и короткий пробный запуск прежде, чем считать конфигурацию подходящей.

Небольшой опыт

Запустите одинаковый вход после прогрева и измерьте отдельно чтение документа, поиск, prefill и decode. Затем удвойте только длину входа и только длину выхода. Если меняется главным образом TTFT, исследуйте prefill и поиск; если скорость токенов падает при росте числа запросов, смотрите KV, batch и bandwidth. FlashAttention⁴ служит напоминанием: перенос данных можно уменьшить алгоритмом, не покупая новый ускоритель.

Не путать измерения

Пиковая VRAM определяет размещение. Утилизация GPU не показывает, сколько полезных токенов завершено. Скорость SSD в большом последовательном тесте не равна задержке мелких чтений.

Тест стабильности

Прогрев, серия длинных запросов, загрузка индекса и параллельный OCR. Записать температуру, частоты, память, p95 задержки и ошибки.

Диагноз

Один изменённый параметр за опыт: длина входа, число одновременных генераций или местоположение весов.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

4. Dao, T. и др. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.

FlashAttention: точное внимание с учётом обмена между уровнями памяти

Разбиение вычисления attention на блоки уменьшает движение данных между HBM и быстрой памятью GPU. Это пример ускорения без изменения математической цели attention; эффект зависит от формы задачи и реализации.

Что читать. Постановка IO-aware attention и раздел Algorithm; объяснить, какие промежуточные данные перестают записываться в HBM.

Порядок чтения. Сначала CUDA и Best Practices для пути данных. Затем Roofline, чтобы построить гипотезу об узком месте. FlashAttention использовать как контрольный пример алгоритмической альтернативы. После этого измерить конкретный этап своего процесса.

2. Сколько памяти требует модель

Оценка вместимости начинается с весов, но заканчивается кэшем, буферами и фактическим распределением запросов. Число параметров не равно расходу VRAM.

Нижняя граница для плотных весов

Если плотная модель имеет P параметров и каждый занимал бы ровно b бит, сырые веса потребовали бы $W_0 = Pb/8$ байт. Для $P = 8$ миллиардов и $b = 4$ получаем 4 миллиарда байт, или около 3,73 GiB. Это нижняя оценка, а не гарантированный размер файла: схемы квантования хранят масштабы и другие метаданные по блокам; отдельные слои могут оставаться в другом формате.

Нельзя смешивать GB и GiB: первый равен 10^9 байт, второй 2^{30} . Сравнение «4 GB весов» с «4 GiB свободной памяти» без учёта runtime и кэша ошибочно. Для MoE различают общее число параметров, которые надо хранить или подгружать, и число активных параметров при обработке токена. Слова «активно 8B» не означают, что достаточно памяти как для плотной модели 8B.

Кэш внимания

В обычном decoder-only Transformer без оконной схемы нижнюю оценку полного KV-кэша можно записать так:

$$K = 2LH_{KV}DSBs.$$

Здесь L — слои, H_{KV} — число голов ключей/значений, D — размер головы, S — удерживаемые токены одной последовательности, B — число активных последовательностей, s — байт на элемент. Множитель 2 относится к K и V. Для $L = 32$, $H_{KV} = 8$, $D = 128$, $S = 4096$, $B = 1$, $s = 2$ выходит 536,870,912 байт, то есть 0,5 GiB. Двадцать таких последовательностей дают 10 GiB до расходов менеджера кэша.

Документация о кэше¹ помогает проверить, что эта формула применима. У GQA в H_{KV} подставляются KV-головы, а не query-головы. Скользящее окно, MLA, сокращение кэша, prefix sharing и разные точности меняют картину. Нужна конфигурация именно выбранной архитектуры и версия runtime, а не одна универсальная таблица по числу параметров.

Учебные числа

8B, 4 бита: $4 \cdot 10^9$ байт сырых весов $\approx 3,73$ GiB.

Полный KV

32 слоя; 8 KV-голов; 128 на голову; 4096 токенов; 2 байта: 0,5 GiB на последовательность, 10 GiB на двадцать.

Проверь архитектуру

GQA, MLA, окно внимания и точность KV. Длина контекста в карточке модели не является заранее выделенным кэшем во всех runtime.

Рабочая память и пик

Файл модели, занятая устройством память и память под один запрос — разные числа. При загрузке выделяются буферы вычислений, служебные структуры и иногда копии тензоров. Prefill с длинным входом может иметь отдельный пик временной памяти. При обслуживании нескольких запросов менеджер хранит свободные блоки и распределяет KV не так, как непрерывный массив в формуле.

PagedAttention³ разбивает логический кэш на блоки, уменьшая потери из-за выделения и позволяя делить общие префиксы. Значит, нижняя формула по токенам сохраняет смысл как полезные данные, но фактическая выделенная память зависит от размеров блоков и политики планировщика. Сначала измеряют холодный запуск, затем несколько реальных последовательностей с разными длинами и остановкой генерации.

Когда веса не входят в VRAM

Big Model Inference² показывает размещение весов по устройствам и выгрузку. Перенос части слоёв в RAM иногда позволяет запустить checkpoint при меньшей VRAM. Но рост времени ответа может сделать эту конфигурацию непригодной; надо измерить throughput и задержку на длительной генерации. Диск как резервное хранилище весов ещё сильнее отличается от быстрой памяти ускорителя.

Реальные форматы проверяют по документам runtime, например llama.cpp⁴. «Q4» не эквивалентно $P/2$ байт на диске: тип блоков и сохранённые в другой точности тензоры добавляют накладные данные. Результат квантования оценивают и по качеству ответа, особенно для языков и задач собственного корпуса.

Три измерения одной гипотезы

Выберите точный checkpoint и запишите фактический размер файла, объём VRAM после загрузки и пик при запросе. Повторите при 512 и 4096 входных токенах, затем при одном, четырёх и двадцати активных запросах. Сравните наблюдения с $W_0 + K$, перечислите источники расхождения, проверьте отсутствие скрытой выгрузки. Лишь после такого опыта можно сказать, что данная модель помещается и работает в выбранном режиме.

Бюджет на GPU

Веса + KV + буферы вычислений + allocator + пик prefill + запас. Ни один компонент нельзя автоматически принять за ноль.

Три проверки

Файл

Размер GGUF или safetensors.

Покой

VRAM после загрузки.

Нагрузка

Пик и p95 на целевых длинах и конкурентности.

Ограничение

Схемы KV архитектурно различаются. Все числа здесь — учебный расчёт, не замер конкретного checkpoint.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Caching*. Документация.

Кэширование ключей и значений при генерации

Показаны варианты KV-кэша и ограничения отдельных реализаций: динамическое размещение, статический кэш и перенос. Документация помогает проверить, применяется ли простая формула полного кэша к выбранной архитектуре.

Что читать. Introduction; Cache Types; Offloaded Cache. Выписать поддерживаемый тип для конкретного checkpoint.

2. Hugging Face (документация, 2026), *Big Model Inference*. Документация.

Инференс моделей, не помещающихся целиком в ускоритель

Разъясняет dispatch весов между GPU, RAM и диском. Такая выгрузка может сделать запуск возможным, но пропускная способность пути передачи меняет практическую скорость.

Что читать. Big Model Inference и описание device map; различить факт загрузки и измеренную скорость.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

4. ggml-org (документация, 2026), *Model files and quantization in llama.cpp*. Документация и код.

Файлы моделей и квантование в llama.cpp

Описаны форматы моделей и варианты запуска. Обозначение четырёхбитной квантизации относится к конкретной схеме блоков и метаданных, поэтому размер файла не обязан равняться идеальному четырём битам на параметр.

Что читать. Разделы о GGUF, quantization и loading; сравнить размер файлов одинакового checkpoint в двух форматах.

Порядок чтения. Начать с видов KV-кэша, затем изучить загрузку большой модели. PagedAttention объясняет распределение при конкуренции, документация llama.cpp — фактический формат весов. На выбранной модели сопоставить формулу с пиковой памятью.

3. Обучение и дообучение: от узла к кластеру

Обновление весов добавляет градиенты, состояния оптимизатора и сохранённые активации. Экономия памяти меняет обмен данными, время шага и результат обучения.

Почему память инференса нельзя переносить на обучение

При инференсе хранятся веса, KV и рабочие буферы. Полное обучение дополнительно требует градиенты, состояния оптимизатора, активации для обратного прохода и временные тензоры. Для учебной плотной модели с 8 миллиардами параметров только 16-битные веса заняли бы около 14,9 GiB; сумма с градиентами, мастер-копиями и состояниями оптимизатора может быть намного больше. Точное число определяется выбранным оптимизатором и форматом, поэтому правило «столько-то байт на параметр» без этих условий опасно.

Увеличение длины последовательности и числа примеров в микробатче повышает расход активаций. Gradient accumulation уменьшает размер одного шага на устройстве, сохраняя больший эффективный batch, но добавляет проходы и не устраняет память весов. Activation checkpointing пересчитывает часть операций на обратном проходе и меняет время на память. Прежде чем покупать устройство, определите, какие параметры действительно требуется обновлять.

Полный шаг, адаптер и квантованная база

Полное дообучение обновляет все веса и требует соответствующих градиентов и оптимизатора. LoRA замораживает базу и обучает низкоранговые добавки к выбранным слоям; активации и проход через базу всё равно нужны. QLoRA⁴ размещает замороженную базу в квантованном виде и обучает адаптеры. Это позволяет снизить расход под веса и обучаемые состояния, но не делает активации или пик временных буферов бесплатными.

Для задачи документов сравните три вопроса: можно ли достичь цели правильным извлечением без обучения; если нет, поможет ли небольшой адаптер; если нужен полный сдвиг модели, хватит ли данных и вычислительного бюджета. Улучшение обучающей потери не доказывает качество на новых документах. Создайте отложенный набор до запуска и зафиксируйте метрики цитирования и задач пользователя.

Пять корзин обучения

Веса; градиенты; оптимизатор; активации; временные тензоры и коммуникация.

LoRA/QLoRA

Меньше обучаемых параметров не означает отсутствие вычислений через базу.

Первый вопрос

Нужен ли fine-tuning для желаемого эффекта или ошибка находится в данных и поиске?

Запись опыта

Данные, checkpoint, precision, sequence length, microbatch, accumulation, peak VRAM, шаги/с, качество.

Несколько GPU и стоимость синхронизации

Простейший data parallelism держит реплику модели на каждом GPU и объединяет градиенты. Если один экземпляр не входит в память одного устройства, дополнительные реплики сами по себе не помогают. FSDP² и ZeRO³ разделяют параметры, градиенты и состояния оптимизатора между устройствами; необходимые данные собираются по мере шага. Общая доступная память растёт, но ускорители начинают ждать обмена.

NCCL² описывает коллективные операции, например all-gather и reduce-scatter. Их стоимость зависит от топологии: связь внутри одного узла и между узлами имеет разную полосу и задержку. Tensor и pipeline parallelism добавляют другие границы обмена и распределения работы. Для кластера нужны скорость хранилища checkpoint, сеть, управление задачами, мониторинг и пробное восстановление, а не только число ускорителей.

Сквозная экономика эксперимента

План полного обучения включает оценку токенов и шагов, времени шага на прототипе, объёма checkpoint, частоты сохранения и вероятного повторного запуска. Если 8 GPU повышают throughput не в 8 раз, разница может уходить в обмен, дисбаланс или чтение данных. Масштабирование лучше измерять в *полезных* токенах за час при одинаковом качестве и плане обучения.

В локальной задаче сначала возьмите малую открытую модель и проведите один воспроизводимый шаг. Измерьте пик памяти и время для полного обновления, LoRA и QLoRA на одинаковой выборке. Затем отдельно оцените качество, поскольку самый компактный режим может иметь иные ограничения. Если обучение выполняется раз в полгода, удалённые вычисления на короткий период могут быть практичнее постоянно включённого кластера; такое решение требует учесть локальность данных.

Что сохранять после сбоя

Одних весов недостаточно для точного возобновления. Нужны состояния оптимизатора, планировщика, генераторов случайности, позиция в данных и сведения о версии кода. Проверьте восстановление в отдельном запуске, иначе высокая скорость обучения на бумаге может обернуться повторением всего эксперимента. Для адаптеров фиксируйте и точную ревизию базовой модели: без неё файл добавок неоднозначен.

Два разных роста

Шардирование помогает уместить состояние. Оно не обещает линейного ускорения: all-gather и запись checkpoint занимают время.

Тест масштаба

Одинаковые данные: 1 GPU, затем 2. Зафиксировать память каждого, шаги/с, обмен и качество.

Восстановление

Проверить checkpoint после намеренного прерывания — отдельная часть протокола обучения.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. PyTorch (документация, 2026), *Getting Started with Fully Sharded Data Parallel (FSDP2)*. Документация.

Полное шардирование параметров, градиентов и оптимизатора

Разделяет репликацию модели и шардирование состояний обучения. Описаны коллективные операции, временное восстановление весов и компромисс между памятью и коммуникацией.

Что читать. Overview; How FSDP2 works; all-gather и reduce-scatter. Схематично изобразить один шаг на двух GPU.

2. NVIDIA (документация, 2026), *NCCL Developer Guide*. Документация.

Коллективные операции NCCL при распределённом обучении

Показывает коллективные операции и коммуникационные топологии между GPU. Для multi-GPU конфигурации скорость межсоединения и доступный набор маршрутов важны наряду с суммой VRAM.

Что читать. Overview и Collective Operations; сравнить all-reduce, all-gather и reduce-scatter для градиентов.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Rajbhandari, S. и др. (2020), *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC20.

ZeRO: уменьшение памяти обучения разделением состояний

Авторы разбирают память оптимизатора, градиентов и параметров и этапы их разделения между процессами. Выигрыш памяти оплачивается обменом данными и зависит от топологии кластера.

Что читать. §2 о памяти состояний; §3 этапы ZeRO; проверить, что именно шардируется на каждом этапе.

4. Dettmers, T. и др. (2023), *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS.

QLoRA: эффективное дообучение квантованных LLM

Замороженная квантованная база сочетается с обучаемыми низкоранговыми адаптерами. Работа объясняет экономию памяти и цену активаций; число параметров адаптера само по себе не определяет весь пик VRAM.

Что читать. §3 QLoRA, quantization и paged optimizers; §4 эксперименты. Восстановить состав памяти для одного шага.

Порядок чтения. FSDP2 вводит распределённый шаг, NCCL — его операции обмена. ZeRO даёт модель памяти шардирования. QLoRA показывает другой путь снижения расхода: заморозить базу и обучать адаптер. Сопоставить все четыре подхода на одной задаче.

4. Инференс: от одного запроса к сервису

Скорость ответа складывается из поиска, обработки входа, генерации и ожидания в очереди. Пиковые токены в секунду не описывают поведение сервиса под нагрузкой.

Две фазы генерации

При prefill модель обрабатывает входную последовательность и создаёт начальное KV-состояние. При decode она последовательно генерирует токены, дополняя это состояние. Большой вход влияет прежде всего на время первого токена (TTFT), длинный ответ — на длительность decode. Поисковая стадия и очередь дополняют задержку, которую видит пользователь. Если в отчёте указан только средний throughput, трудно понять, почему интерактивный ответ медленный. Для учебного RAG-запроса измерьте отдельно поиск и reranking, токенизацию, prefill, первый токен и следующие 128 токенов. Удвоение числа приложенных фрагментов меняет вход и может улучшить либо ухудшить качество; это не бесплатная настройка. На том же GPU одновременные запросы могут улучшить суммарную загрузку, но увеличить ожидание и KV-память.

Планировщик и память

vLLM¹ служит примером сервера, где политика пакетирования и выделения KV связаны. PagedAttention³ уменьшает потери при переменных длинах запросов и поддерживает совместное использование префиксов. Однако блоки кэша конкурируют с весами и временными буферами. При слишком длинных входах или большом числе активных генераций сервер может вытеснять, пересчитывать либо отклонять работу в зависимости от настроек и версии.

Полезно отличать число зарегистрированных пользователей, число запросов в секунду и *одновременно активных последовательностей*. Двадцать сотрудников могут обращаться по очереди; двадцать параллельных длинных генераций создают другой пик памяти. Для определения этого пика соберите распределение поступления запросов, а не умножайте число учётных записей на максимальное контекстное окно.

Метрики с единицами

TTFT — время до первого токена. TPOT или ITL описывают темп выдачи следующих токенов; конкретное определение нужно записать. End-to-end latency включает все стадии. Goodput можно определить как число завершённых запросов, уложившихся в заданные ограничения качества и задержки за единицу времени. p95 и p99 показывают хвост, который скрывается за средним. Для справедливого сравнения фиксируют точную модель, длины входа и выхода, интенсивность, качество и версии ПО.

Видимая задержка

Очередь + поиск + prefill + decode + постобработка. Измеряйте составляющие отдельно.

Метрики

TTFT: до первого токена. TPOT: среднее время следующего. p95: 95-й процентиль распределения.

Нагрузка

Пользователей $\neq$ активных запросов. Активных запросов $\neq$ максимум резервирования во всех планировщиках.

Как ставить нагрузочный опыт

Benchmark CLI vLLM² задаёт измеримый генератор запросов. Сначала прогрейте систему и отдельно измерьте одиночный запрос с 512 входными и 128 выходными токенами. Повторите для входа 4096 и ответа 512. Затем задайте частоту поступления и сравните p50/p95 TTFT, TPOT, число завершений, пик VRAM и ошибки. Синтетические токены полезны для контроля длины, но реальный корпус может давать другие префиксы, распределение и поиск.

Изменяйте один параметр за опыт: batch, число параллельных последовательностей, размер блоков KV либо режим offload. Иначе нельзя сказать, из-за чего изменилась скорость. Сохраните конфигурацию и журнал запуска: «на этой машине получилось 42 токена/с» без режима нагрузки и длины текста не является переносимым результатом.

Разделять prefill и decode или нет

DistServe⁴ исследует отдельные группы GPU для фаз prefill и decode. Если длинные входы мешают выдаче токенов другим запросам, разделение может улучшить соблюдение целевых задержек. При этом KV нужно передать между группами, и дорогое межсоединение входит в стоимость решения. Для локальной станции с одним GPU такой путь обычно не относится к ближайшему выбору; для многопользовательского сервера становится исследовательским вариантом.

Другие варианты — отдельные реплики под длинные и короткие запросы, ограничение длины, очередь с приоритетом, динамическое пакетирование. Их сравнивают при одинаковом качестве и полной цене. Более короткий контекст может уменьшить расход, но терять нужный фрагмент. Более сильное квантование может улучшить вместимость, но изменить ответы. Аппаратное решение невозможно отделить от политики сервиса.

Когда инцидент похож на нехватку железа

OOM при загрузке, OOM только на длинных запросах и рост p95 без OOM — разные случаи. В первом проверяют веса и формат; во втором KV и пик prefill; в третьем очередь, CPU, сеть, поиск и частоты устройств. Отчёт об обслуживании должен разделять эти симптомы. Покупка GPU оправдана после того, как протокол показал узкое место, которое дополнительные ресурсы способны устранить.

Матрица опыта

Вход: 512/4096. Выход: 128/512. Конкурентность: 1/4/20. Равные checkpoint, runtime и критерий качества.

Остановить сравнение

Если новая конфигурация нарушает требование качества или ссылки на документы, её более высокий throughput не даёт победы.

Диагностика

OOM весов; OOM KV; медленный prefill; медленный decode; ожидание в очереди — пять разных причин.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. vLLM (документация, 2026), *vLLM Documentation*. Документация и код.

Сервер инференса vLLM и его планировщик

Объединяет планирование запросов, управление KV-памятью и запуск сервера. Конкретные параметры зависят от версии; исходный runtime необходимо фиксировать вместе с измерениями.

Что читать. *Serving* и *Configuration*; проследить вход запроса до выделения KV-блоков и ответа.

2. vLLM (документация, 2026), *Benchmark CLI*. Документация.

Измерение времени первого токена и времени между токенами

Документирует параметры генератора нагрузки и метрики TTFT, TPOT, throughput и задержки. Сопоставление машин требует одинаковой модели, распределения длин и интенсивности поступления запросов.

Что читать. *Benchmark Serving* и определения метрик; составить команду опыта с воспроизводимыми входными длинами.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

ФРОНТИР ИССЛЕДОВАНИЙ

4. Zhong, Y. и др. (2024), *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI.

DistServe: раздельное исполнение prefill и decode

Предлагается разместить фазы prefill и decode на разных группах GPU, чтобы уменьшить помехи и отдельно выбирать их параллелизм. Передача KV-состояния тоже имеет цену, поэтому результат зависит от нагрузки и сети.

Что читать. §2 анализ помех; §3 архитектура; §4 оптимизация размещения и измерения goodput.

Порядок чтения. Документация vLLM описывает сервер, PagedAttention — управление кэшем. Затем изучить определения метрик в Benchmark CLI и воспроизвести опыт. DistServe читать после понимания фаз, оценивая стоимость передачи состояния.

5. Данные, индексы и базы для RAG

Надёжная документная система тратит память и диск на исходники, тексты, векторы, индексы, метаданные и версии. Качество поиска измеряется отдельно от генерации.

От файла до проверяемого фрагмента

Корпус PDF и таблиц поступает на диск. Извлечение и OCR создают текст с привязкой к странице и документу. Чанкирование даёт фрагменты, эмбединги — векторы; база хранит их идентификаторы, разрешения, версию и путь к исходнику. Если документ обновлён, старый вектор может продолжать отвечать на вопросы, пока операция переиндексации не выполнена согласованно.

Introduction to Information Retrieval¹ напоминает, что релевантность надо проверять на наборе вопросов. Для RAG пригодна параллельная оценка: попал ли нужный фрагмент в top-k, можно ли процитировать правильную страницу, и сохраняется ли ответ при изменении документа. Хороший ответ генератора не доказывает хорошего индекса: модель могла угадать, опираясь на уже известный ей факт.

Оценка сырых векторов

При N векторов размерности d , хранимых в s байтах на компонент, нижняя граница равна $V_0 = Nds$. Для 150,000 фрагментов по 1024 компонента FP32 получим 614,400,000 байт, приблизительно 0,572 GiB. Это только числа векторов. Текст, метаданные, индексные связи, структуры БД, WAL и резервная копия добавляются отдельно. Если размерность или формат меняется, весь расчёт надо повторить. pgvector² позволяет сравнивать точный поиск с HNSW и IVFFlat внутри PostgreSQL. Точный поиск служит эталоном recall на маленькой проверочной выборке. Приближённый индекс ускоряет запросы ценой дополнительной памяти и вероятности пропуска соседа. SQL-фильтр по доступу и версии может резко уменьшить выборку и изменить эффективный план запроса; векторный бенчмарк без такого фильтра не описывает реальную задачу.

Где должен стоять SSD

Размер файла векторов не равен диску под БД. Учтите исходники, извлечённый текст, индексы, временное место перестроения, транзакционные журналы и копии. Для горячего индекса важны RAM и характер доступа; для редкого архива — ёмкость и скорость восстановления. Не размещайте единственную резервную копию на том же отказавшем устройстве.

Сырой векторный массив

$150000 \times 1024 \times 4$ байта = 0,572 GiB. Индекс и текст не учтены.

Два качества

Recall поиска по фрагментам и качество ответа с корректной цитатой — разные метрики.

Версия

Каждый вектор связан с ревизией текста, страницей и разрешениями. Удаление источника должно удалять или инвалидировать производные данные.

Граф индекса и граф знаний

HNSW³ строит граф близости между векторами для приближённого поиска. Рёбра отражают геометрию эмбедингов и путь поиска кандидатов. Это не граф предметных фактов: отношение «сотрудник отвечает за проект» надо хранить и проверять отдельно. Рост параметров построения и поиска меняет компромисс между recall, временем и памятью; универсального значения для каждого корпуса нет. Сравнение поиска ставят на фиксированном списке вопросов с размеченными нужными фрагментами. Измеряют recall@k относительно эталона, р95 времени с учётом фильтров и пик RAM при построении и обновлении. Результат генерации проверяют отдельно: правильный фрагмент может быть извлечён, но проигнорирован моделью; плохой фрагмент может получить уверенную ссылку. Реранжирование добавляет ещё одну модель и свой бюджет CPU/GPU.

Обновление и восстановление

Документация PostgreSQL по PITR⁴ объясняет согласованную резервную копию и WAL. Но восстановление одной таблицы векторов не возвращает автоматически те PDF и модели эмбедингов, из которых она была построена. Для нашей системы запишите version ID исходника, конфигурацию OCR, правила чанкирования, идентификатор модели и версию индекса. После изменения любого из них переиндексация должна иметь явный статус.

Учебный тест отказа: снимите резервную копию, добавьте и измените два документа, удалите один, затем восстановитесь на отдельной машине. Проверьте не только число строк, но и доступность исходных страниц, разрешения и ответы по трём контрольным вопросам. Восстановление, которое ни разу не испытывали, остаётся предположением.

Системный бюджет

Рассчитывайте отдельно (1) исходники и архив, (2) рабочий корпус с индексом и журналом, (3) модели поиска и генерации с их оперативной памятью. Это помогает понять, когда добавление SSD решает проблему, а когда нужен объём RAM или GPU. Для крупного многопользовательского корпуса учитывайте сетевую передачу, резервирование БД и время построения нового индекса без остановки сервиса.

HNSW — что за граф

Связи между близкими векторами, не утверждения о мире. Его размер и полноту надо измерять на своём корпусе.

Тест восстановления

База + документы + версии OCR/embedding + разрешения. Восстановить на другом узле и задать контрольные вопросы.

Три корзины

Исходники; индекс и метаданные; модели и обслуживание запросов.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Manning, C. D.; Raghavan, P.; Schütze, H. (2008), *Introduction to Information Retrieval*. Учебник.
Введение в информационный поиск
Объясняет инвертированные индексы, ранжирование и оценку поиска. Качество RAG начинается с выбора релевантных фрагментов, а не с объёма VRAM генератора.
Что читать. Главы 1, 6 и 8: индекс, ранжирование и оценка; задать тестовый набор запросов.
2. pgvector (документация, 2026), *pgvector: Open-source vector similarity search for Postgres*. Документация и код.
Векторный поиск в PostgreSQL
Описаны точный поиск, HNSW и IVFFlat, типы векторов, индексы и SQL-фильтрация. Выбор индекса отражается на RAM, размере диска, времени построения и качестве поиска.
Что читать. Exact Search; HNSW; IVFFlat; Filtering. Замерить recall относительно точной выдачи после фильтра.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Malkov, Y. A.; Yashunin, D. A. (2016), *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. Исследовательская статья.
HNSW: приближённый поиск по иерархическому графу близости
Иерархический граф позволяет быстро находить кандидатов, меняя время и память на достигаемую полноту. Векторный индекс хранит связи между точками помимо самих чисел вектора.
Что читать. §3 построение и поиск; §4 эксперименты. Сравнить стоимость индекса и recall с точным поиском.
4. PostgreSQL Global Development Group (документация, 2026), *Continuous Archiving and Point-in-Time Recovery (PITR)*. Документация.
Непрерывное архивирование и восстановление базы на момент времени
Резервная копия и журнал WAL позволяют восстановить согласованное состояние. Для RAG надо также согласовать версию исходного документа, фрагментов, эмбеддингов и индекса.
Что читать. Setting Up WAL Archiving; Making a Base Backup; Recovering. Провести пробное восстановление на отдельном узле.

Порядок чтения. Учебник по IR задаёт качество поиска. pgvector показывает исполнимые альтернативы, HNSW — устройство приближённого индекса. PITR завершает путь: восстановить не только записи, но и согласованные производные данные.

6. Локальные модели: от SLM до LLM

Локальный запуск определяется совместимостью checkpoint, формата и runtime с устройством. Размер модели, квантизация и разгрузка слоёв выбираются вместе с качеством и задержкой.

Выбрать конкретную модель

Начинают не с абстрактного «7В», а с точного идентификатора модели и ревизии. Проверяют задачу, языки, размер контекста, лицензию, известные ограничения, формат файлов и tokenizer. Карточка модели¹ — отправная точка, но заявленная способность не заменяет собственный тест на документах. Небольшая SLM может быть достаточной для классификации или извлечения, тогда как длинный синтез по нескольким источникам предъявляет другие требования.

Для нашего проекта полезно разделить роли: OCR, embedding, reranker и генератор не обязаны быть одним и тем же checkpoint. Их можно запускать последовательно на одном устройстве, если требуемая задержка позволяет выгрузку, или держать постоянно, если потоки запросов пересекаются. Сумма файлов на диске и одновременный пик памяти — разные величины.

Три вопроса совместимости

Первый: поддерживает ли runtime архитектуру, tokenizer, chat template и нужные модальности. Второй: поддерживает ли backend данного устройства операции, формат весов и точность. Третий: помещаются ли веса, KV и рабочие буферы при целевом контексте. llama.cpp² показывает практический локальный маршрут CPU/GPU с GGUF и сервером; его поддержка конкретных новых моделей проверяется по коду и пробному запуску, а не по общему названию проекта.

Предварительный отбор может выглядеть так: малая модель и CPU для проверки процесса; более крупная квантованная модель для одиночного GPU; серверный runtime для нескольких параллельных запросов. Это сценарии испытания, не пороги по числу миллиардов параметров. Четырёхбитное представление меняет память и иногда качество. Частичный offload меняет достижимую скорость. Отдельно запишите длину входа и ответа, потому что «контекст 128k» может быть программно доступен и аппаратно неприемлем.

Данные локальны только при полном контуре

Локальный генератор не делает систему локальной, если OCR, embedding или телеметрия отправляют документы во внешний сервис. Проверьте каждый процесс, обновления, загрузку весов, лицензии и хранение журналов. В паспорте машины должны быть и политика резервных копий, и возможность восстановить окружение без сети, если этого требует задача.

Точный checkpoint

ID + ревизия + лицензия + tokenizer + chat template + формат весов.

Четыре процесса

OCR, embedding, reranker, генератор. Запишите, какие из них работают одновременно.

«Запустить»

Поддержка операций; выделение памяти; правильный ответ; нужная скорость — четыре независимые проверки.

Разные программные экосистемы

У GPU NVIDIA обычно проверяют CUDA, compute capability, драйвер и соответствующую сборку приложения. Для AMD матрица ROCm³ сопоставляет устройство и версии ОС, фреймворка и runtime; поддержка обозначения серии не равна проверке точной карты и модели. На Apple Silicon MLX⁴ использует общую память CPU/GPU. Это снимает некоторые расходы копирования, но не превращает системную RAM в неограниченную VRAM: ёмкость и bandwidth остаются конечными, а другие процессы тоже расходуют ресурс.

CPU-only полезен как baseline и иногда достаточен для малой модели и низкой частоты запросов. При выборе CPU смотрят на производительность одного потока, векторные инструкции, bandwidth RAM и время полной генерации. Для ноутбука важны также ограничения мощности и охлаждения: краткий бенчмарк не описывает час непрерывной работы. Для дискретного GPU проверьте физическую совместимость, питание и сколько слоёв runtime действительно положил на ускоритель.

От загрузки к воспроизводимому запуску

Сохраните hash или ревизию файлов, команду запуска, лимит контекста, размер batch, число GPU-слоёв либо схему размещения, версии драйвера и runtime. Прогрейте модель. Проведите три набора вопросов: короткие факты, длинные документы и случаи, когда нужно признать отсутствие ответа. Сравните точность цитат и выходов в исходной и квантованной версии, затем скорость и память.

Если квантованная модель влезает, но результат ухудшается на обязательных задачах, выбор не пройден. Если точная модель работает лишь с выгрузкой в RAM и длительное ожидание недопустимо, она тоже не подходит. Иногда модель меньшего размера с лучшим retrieval побеждает более крупную локальную модель при том же бюджете; это проверяется на корпусе, а не предполагается по размерам.

Путь к собственному дообучению

Для локального LoRA сначала зафиксируйте базовый checkpoint, обучающие и отложенные данные, лицензию и формат адаптера. Оцените память по предыдущей главе, сделайте один пробный шаг и измерьте качество после адаптации. Убедитесь, что runtime умеет применить адаптер или что процедура слияния даёт проверенный артефакт. Обучение и запуск часто используют разные стеки — совместимость результата нужно проверить отдельно.

Маршруты запуска

CPU baseline; дискретный GPU; общая память Apple Silicon; сервер для конкурентных запросов. Сравнивать по одной задаче.

Четыре испытания

Загрузка, пик памяти, качество на своём наборе, p95 задержки после прогрева.

Запишите версии

После обновления драйвера или backend результат может измениться: это новый опыт.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Model Cards*. Документация.

Карточки моделей: лицензия, назначение и ограничения

Карточка связывает веса с задачей, происхождением, условиями использования и известными ограничениями. Перед локальным запуском нужно установить точный checkpoint, шаблон диалога и применимую лицензию.

Что читать. Overview и metadata; прочитать карточку именно выбранной ревизии, затем сравнить с конфигурацией tokenizer.

2. ggml-org (документация, 2026), *Llama.cpp*. Документация и код.

Llama.cpp: локальный запуск и сервер моделей

Проект показывает локальный запуск CPU/GPU, частичную выгрузку, сервер и параметры контекста. Наличие backend не гарантирует поддержки каждой новой архитектуры или одинаковой скорости.

Что читать. README, build instructions и server README; запустить один точный checkpoint с фиксированной версией.

3. AMD (документация, 2026), *ROCm Compatibility Matrix*. Документация.

Матрица совместимости ROCm

Матрица сопоставляет устройства, операционные системы и версии программного стека. Проверять нужно конкретные GPU и runtime, а не только надпись о наличии поддержки ROCm.

Что читать. GPU, OS and framework support для выбранной версии; отметить проверенные и неподтверждённые сочетания.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. Apple ML Explore (документация, 2026), *MLX*. Документация и код.

MLX: вычисления на Apple Silicon

Описывает стек для CPU и GPU Apple Silicon с общей памятью и собственным API. Общая память упрощает размещение, но не отменяет ограничения ёмкости, bandwidth или конкуренцию процессов.

Что читать. README и документация о device/array; измерить память приложения и скорость устойчивой генерации.

Порядок чтения. Прочитать карточку выбранной модели, затем документацию Llama.cpp для воспроизводимого CPU/GPU запуска. ROCm и MLX читать по соответствующему устройству. Последним шагом провести собственный тест на одних и тех же вопросах.

7. Выбор компонентов и контрольный практикум

Итог — спецификация, в которой каждому компоненту соответствует измеренное требование. Решение принимают по качеству, p95 задержке, восстановлению и полной цене.

Заполнить паспорт, прежде чем выбирать детали

Для учебного проекта запишите: ревизии всех моделей; пять тысяч PDF и таблиц или реальный размер собственного корпуса; число фрагментов; распределение входа и выхода; целевую конкурентность; требования к ссылкам и защите; окно обслуживания. Оцените W_0 , KV и сырые векторы, добавив явно помеченный запас для runtime, индекса и роста. Затем измерьте текущий baseline хотя бы на малой модели: без него трудно судить, что даст новая покупка. Разбейте спецификацию по компонентам. GPU: поддержка численного формата и ядер, VRAM, bandwidth, устойчивость под нагрузкой. CPU: парсинг, очередь, токенизация, поиск и способность кормить GPU. RAM: корпус, индекс, offload и рабочие процессы. SSD: исходники, модели, WAL, временное переиндексирование и копии. Материнская плата: PCIe, слоты RAM, накопители и физическое размещение. БП и охлаждение: стабильная длительная работа. Сеть: удалённые пользователи, backup или межузловой обмен. Не складывайте ёмкость разных уровней памяти как если бы это был один быстрый пул.

Аппаратная совместимость — проверяемое дерево

Документация о compute capability¹ помогает установить поддерживаемые возможности NVIDIA GPU. Для AMD и Apple проверка строится по их собственным матрицам из предыдущей темы. Далее подтверждают ОС, драйвер, сборку runtime, точный checkpoint и операцию. Наконец, проверяют физическую посадку карты, разъёмы и тепловой режим. В таблице решения каждая строка имеет статус «проверено запуском», «документировано» или «неизвестно»; неизвестное не превращают в совместимое без испытания.

Порядок покупки

Составьте три класса: имеющаяся машина с малой моделью; одна локальная станция с достаточной памятью и надёжным хранилищем; сервер или временная аренда для большой конкурентности и обучения. Для каждого оцените полную цену владения на период работы. Покупка самого мощного компонента не решает задачу, если ограничение находится в индексе или CPU. Отложите точный список продаваемых SKU до проверки цен и актуальных драйверов на дату покупки.

Паспорт решения

Checkpoint; качество; токены; конкурентность; пиковая память; p95; версии ПО; корпус; backup; полная стоимость.

Компоненты

GPU, CPU, RAM, SSD, плата, питание/охлаждение и сеть. Для каждого указать ограничение и измерение.

Статус совместимости

Проверено запуском; подтверждено документацией; остаётся неизвестным.

Четыре опыта

Первый — память. Предскажите нижний расход для небольшой и более крупной модели при 1, 4 и 20 активных последовательностях. Снимите фактический пик, отличите память allocator от полезных тензоров с помощью PyTorch CUDA Memory³ там, где этот инструмент применим. Назовите, почему наблюдение отличается от формулы.

Второй — скорость. На одной модели и одинаковых входах сравните CPU, GPU и частичный offload. Используйте PyTorch Profiler² или соответствующий профайлер runtime, разделите чтение, поиск, prefill и decode. Замерьте короткие и длинные запросы, TTFT, TPOT, p95, мощность и ошибки. Один запуск без прогрева и указания версии не даёт надёжной сравнительной оценки.

Третий — RAG. На реальном срезе корпуса измерьте время OCR и embedding, размер текста и индекса, точный и приближённый поиск с одинаковыми фильтрами. Проверьте страницы цитат и восстановление после изменения документа. Четвёртый — обучение. На небольшой модели сделайте одинаковый цикл для полного обновления, LoRA и QLoRA; сохраните пик памяти, шаги/с, checkpoint и качество на отложенных примерах.

Чтение бенчмарков и решение

MLPerf Inference⁴ задаёт сценарии и правила сопоставления систем. Пользуйтесь внешними результатами как ориентирами, если совпадают задача, точность и ограничения. Для собственной закупки первичен полный эксперимент: качество не ниже требуемого, p95 в целевом режиме, устойчивость под длительной нагрузкой и стоимость за полезную задачу.

Последний артефакт — таблица кандидатов с ссылкой на протокол запуска и причиной выбора. Если ни одна машина не укладывается, изменяйте измеримое требование: модель, retrieval, квантизацию, число одновременных запросов либо схему размещения. Не объявляйте устройство «подходящим» только потому, что файл модели загружается. Пробелы следующего этапа Атласа — лабораторные испытания конкретных конфигураций и рыночные цены на дату покупки, которые этот выпуск не подменяет.

Четыре артефакта

Паспорт нагрузки; расчёт с допущениями; журнал опыта; таблица решения.

Критерий

Качество и цитаты $\geq$ порога; p95 $\leq$ лимита; восстановление успешно; стоимость приемлема.

Граница выпуска

Числа примеров учебные. Совместимость точного checkpoint и цены SKU требуют испытания и проверки перед покупкой.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA GPU Compute Capability*. Документация.

Вычислительные возможности GPU NVIDIA

Возможности чипа связаны с набором операций и поддержкой форматов. Два ускорителя с одинаковой ёмкостью VRAM могут по-разному исполнять нужные ядра и поддерживаться фреймворком.

Что читать. Таблицы compute capability и feature support; проверить архитектуру своего GPU до установки runtime.

2. PyTorch (документация, 2026), *PyTorch Profiler*. Документация.

Профилирование вычислений и памяти в PyTorch

Профайлер помогает увидеть время операций, перемещение CPU/GPU и выделения памяти. Показатели нужны для проверки гипотезы об узком месте, прежде чем менять аппаратную часть.

Что читать. Profiler API; измерить три одинаковых запроса после прогрева и сопоставить CPU с GPU.

3. PyTorch (документация, 2026), *Understanding CUDA Memory Usage*. Документация.

Понимание расхода памяти CUDA

Инструменты snapshot и память allocator позволяют отделить выделенные тензоры, зарезервированную память и пики. Не каждая внешняя аллокация видна одному инструменту.

Что читать. Recording memory history; Visualizing snapshots; проверить видимость внешних аллокаций.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. MLCommons (документация, 2026), *MLPerf Inference*. Бенчмарк и правила.

MLPerf Inference: сценарии и сопоставимые измерения

Набор задаёт сценарии измерений и ограничения для сравнения систем. Опубликованный результат полезен для ориентира лишь при сопоставимой задаче, модели, точности и настройке системы.

Что читать. Scenarios, rules and results; перечислить, что расходится с собственной нагрузкой.

Порядок чтения. Сначала установить возможности GPU, затем инструментами профилирования и памяти проверить свою систему. MLPerf читать как образец дисциплины сравнения, учитывая несовпадение задачи. Результат — воспроизводимое решение по классу машины.